Subject: ASIC Design Lab

Experiment No: 1

Title: Design, Synthesis, and Analysis of 4-bit Ripple Carry Adder (ASIC Flow)

Aim: To design a **4-bit Ripple Carry Adder (RCA)** using Verilog and VHDL, perform **functional simulation**, **ASIC-oriented synthesis**, and analyze **area, power, and timing**, identifying the **critical path and delay**.

Theory

Ripple Carry Adder consists of cascaded Full Adders.

Each full adder computes:

- Sum = A ⊕ B ⊕ Cin
- Cout = (A·B) + (Cin·(A ⊕ B))

The carry output of each stage *ripples* to the next, making delay proportional to bit-width.

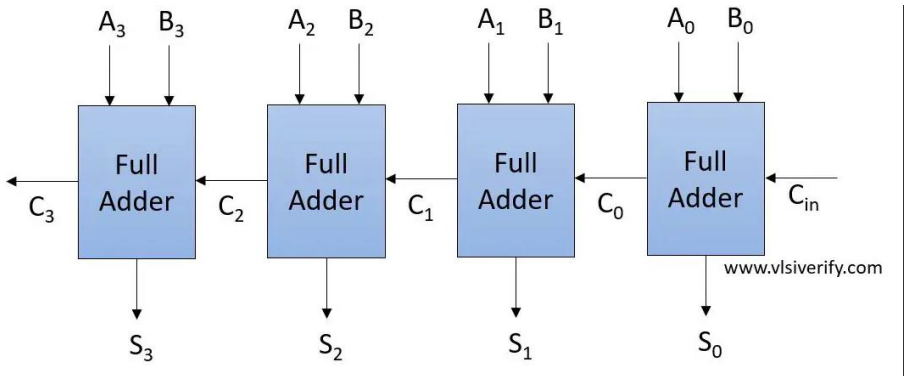
Why RCA is important in ASIC design:

- Simple structure
- Low area
- Predictable timing
- Used as baseline for timing comparison

Drawback:

- Worst-case delay = N × full adder carry delay

Block Diagarm



4-Bit Ripple Carry Adder

Full Adder – Verilog

```
module full_adder (
    input  a,
    input  b,
    input  cin,
    output sum,
    output cout
);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));
endmodule
```

4-bit Ripple Carry Adder – Verilog

```
module ripple_carry_adder_4bit (
    input  [3:0] A,
    input  [3:0] B,
    input        Cin,
    output [3:0] Sum,
    output       Cout
);
    wire c1, c2, c3;
    full_adder FA0 (A[0], B[0], Cin,  Sum[0], c1);
    full_adder FA1 (A[1], B[1], c1,   Sum[1], c2);
    full_adder FA2 (A[2], B[2], c2,   Sum[2], c3);
    full_adder FA3 (A[3], B[3], c3,   Sum[3], Cout);
endmodule
```

Functional Simulation

Test bench (Verilog example)

```
module tb_rca;
   reg  [3:0] A, B;
   reg        Cin;
   wire [3:0] Sum;
   wire       Cout;
   ripple_carry_adder_4bit DUT (A, B, Cin, Sum, Cout);
   initial begin
      A = 4'b0000; B = 4'b0000; Cin = 0;
      #10 A = 4'b0011; B = 4'b0101;
      #10 A = 4'b1111; B = 4'b0001;
      #10 A = 4'b1010; B = 4'b0101;
      #10 $finish;
   end
endmodule
```

Expected Simulation Output

A	B	Cin	Sum	Cout
0000	0000	0	0000	0
0011	0101	0	1000	0
1111	0001	0	0000	1
1010	0101	0	1111	0

Synthesis (ASIC Flow)

Typical ASIC Tools

- Synopsys Design Compiler
- Cadence Genus
- OpenLane (educational)

Synthesis Steps

1. Read HDL
2. Set target library (e.g., 45nm / 28nm standard cells)

3. Apply constraints (clock, input/output delays)
4. Compile and optimize
5. Generate reports

Example Constraints (SDC)

set_clock_uncertainty 0.1

set_input_delay 0.2 -clock clk [all_inputs]

set_output_delay 0.2 -clock clk [all_outputs]

Area, Power, and Timing Analysis

Area (Gate Count Approximation)

Each Full Adder ≈

- 2 XOR
- 2 AND
- 1 OR

For 4-bit RCA:

- XOR gates: ~8
- AND gates: ~8
- OR gates: ~4

➡ **Low area**, minimal cell count

Power Analysis (ASIC)

Power = Dynamic + Static

Dynamic Power

$P = \alpha \cdot C \cdot V^2 \cdot f$

- High switching on carry chain
- Carry ripple increases internal activity

Observation:

RCA has **higher dynamic power** than carry-look ahead for large bit widths

Critical Path & Delay Estimation

Critical Path

Cin → FA0 → FA1 → FA2 → FA3 → Cout

This is the **longest combinational path**.

Delay Estimation

If:

Full Adder carry delay ≈ 120 ps

Then:

Total delay ≈ 4 × 120 ps = 480 ps

Sum MSB also depends on final carry → same path.

Post-Synthesis Simulation

Steps:

1. Generate gate-level netlist
2. Back-annotate delays (SDF)
3. Re-simulate using same testbench

Observations:

- Functional correctness preserved
- Signal transitions show propagation delay
- Glitches visible on intermediate carries

Results Summary

Metric	Observation
Area	Very low
Power	Moderate due to carry switching
Timing	Linear delay with bit width
Critical Path	Carry chain
Scalability	Poor for high-speed designs

Conclusion

The 4-bit Ripple Carry Adder was successfully designed, synthesized, and analyzed using an ASIC-oriented workflow. While it offers simplicity and low area, its linear carry propagation delay limits high-performance applications. RCA serves as a fundamental building block and reference architecture in ASIC digital design.

1. Full Adder – Verilog (Gate-level RTL)

```
module full_adder (
    input  wire a,
    input  wire b,
    input  wire cin,
    output wire sum,
    output wire cout
);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));
endmodule
```

2. 4-bit Ripple Carry Adder – Verilog (Structural)

```
module ripple_carry_adder_4bit (
    input  wire [3:0] A,
    input  wire [3:0] B,
    input  wire       Cin,
    output wire [3:0] Sum,
    output wire       Cout
);
    wire c1, c2, c3;
    full_adder FA0 (
        .a   (A[0]),
```

```
        .b   (B[0]),
        .cin (Cin),
        .sum (Sum[0]),
        .cout(c1)
    );
    full_adder FA1 (
        .a   (A[1]),
        .b   (B[1]),
        .cin (c1),
        .sum (Sum[1]),
        .cout(c2)
    );
    full_adder FA2 (
        .a   (A[2]),
        .b   (B[2]),
        .cin (c2),
        .sum (Sum[2]),
        .cout(c3)
    );

    full_adder FA3 (
        .a   (A[3]),
        .b   (B[3]),
        .cin (c3),
        .sum (Sum[3]),
        .cout(Cout)
    );
endmodule
```

3. Testbench – Verilog (Functional Simulation)

```
module tb_ripple_carry_adder_4bit;

    reg  [3:0] A;
    reg  [3:0] B;
    reg        Cin;
    wire [3:0] Sum;
    wire       Cout;
    ripple_carry_adder_4bit DUT (
        .A(A),
        .B(B),
        .Cin(Cin),
        .Sum(Sum),
        .Cout(Cout)
    );
    initial begin
        $monitor("Time=%0t A=%b B=%b Cin=%b | Sum=%b Cout=%b",
                 $time, A, B, Cin, Sum, Cout);

        A = 4'b0000; B = 4'b0000; Cin = 0;
        #10 A = 4'b0011; B = 4'b0101; Cin = 0;
        #10 A = 4'b1111; B = 4'b0001; Cin = 0;
        #10 A = 4'b1010; B = 4'b0101; Cin = 1;
        #10 A = 4'b0110; B = 4'b0011; Cin = 0;
        #10 $finish;
    end
endmodule
```

Experiment No: 2

Title: DESIGN, SYNTHESIS AND SIMULATION OF 4-BIT PARALLEL ADDER

Aim: To design, simulate, and synthesize a **4-bit parallel adder** using either **Behavioral, Dataflow, and Structural modeling styles in Verilog HDL**, and to analyze **power, area (LUTs/gates), critical timing, and delay** targeting an **ASIC design workflow**.

Software Requirements

- Verilog HDL
- Simulation Tool: ModelSim / Vivado Simulator / Icarus Verilog
- Synthesis Tool: Synopsys Design Compiler / Cadence Genus / Vivado
- Technology Library: Standard Cell Library (ASIC) or FPGA library

Theory

A **4-bit parallel adder** adds two 4-bit binary numbers (A[3:0] and B[3:0]) along with a carry-in (Cin) and produces a 4-bit sum (Sum[3:0]) and carry-out (Cout).

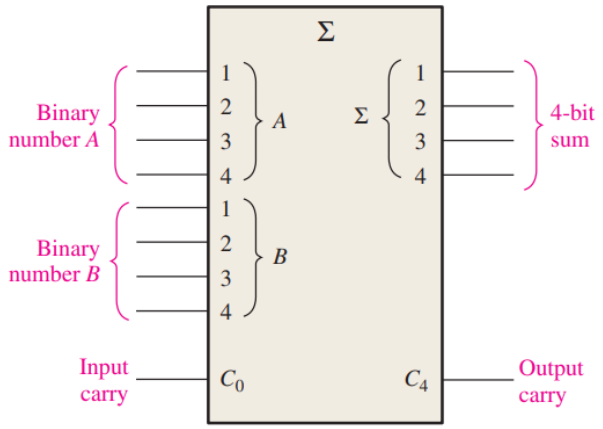
Adder Equation

Sum = A + B + Cin

Modeling Styles

Modeling Style	Description
Behavioral	High-level description using + operator
Dataflow	Continuous assignments using Boolean equations
Structural	Gate / full-adder instantiation

Block Diagram



Verilog HDL Code

Behavioral Modeling	**Dataflow Modeling**	**Structural Modeling**
Full Adder Module		

module adder4_behavioral (
input [3:0] A, B,
input Cin,
output reg [3:0] Sum,
output reg Cout
);
always @(*) begin
{Cout, Sum} = A + B + Cin;
end
endmodule
	module adder4_dataflow (
	input [3:0] A, B,
	input Cin,
	output [3:0] Sum,
	output Cout
	);
	wire [4:0] temp;
	assign temp = A + B + Cin;
	assign Sum = temp[3:0];
	assign Cout = temp[4];
	endmodule
		module full_adder (
		input A, B, Cin,
		output Sum, Cout
		);
		assign Sum = A ^ B ^ Cin;
		assign Cout = (A & B) | (B & Cin) | (A & Cin);
		endmodule

module adder4_behavioral (input [3:0] A, B, input Cin,	module adder4_dataflow (input [3:0] A, B, input Cin,	module adder4_structural (input [3:0] A, B, input Cin,

output reg [3:0] Sum,
output reg Cout
);
always @(*) begin
{Cout, Sum} = A + B + Cin;
end
endmodule
	output [3:0] Sum,
	output Cout
	);
	wire [4:0] temp;
	assign temp = A + B + Cin;
	assign Sum = temp[3:0];
	assign Cout = temp[4];
	endmodule
		output [3:0] Sum,
		output Cout
		);
		wire c1, c2, c3;
		full_adder FA0 (A[0], B[0], Cin, Sum[0], c1);
		full_adder FA1 (A[1], B[1], c1, Sum[1], c2); full_adder FA2 (A[2], B[2], c2, Sum[2], c3);
		full_adder FA3 (A[3], B[3], c3, Sum[3], Cout);
		endmodule

Advantages: Minimal code Fast simulation Used for algorithm verification	**Advantages**: Good balance between abstraction and hardware clarity Easy for synthesis optimization	**Advantages** Closest to hardware Accurate delay and power estimation Preferred for ASIC timing analysis

Testbench

```
module tb_adder4;
   reg  [3:0] A, B;
   reg        Cin;
   wire [3:0] Sum;
   wire       Cout;
   adder4_behavioral DUT (A, B, Cin, Sum, Cout);
   initial begin
      A = 4'b0000; B = 4'b0000; Cin = 0;
      #10 A = 4'b0011; B = 4'b0101; Cin = 0;
      #10 A = 4'b1111; B = 4'b0001; Cin = 0;
      #10 A = 4'b1010; B = 4'b0101; Cin = 1;
      #10 $finish;
   end
endmodule
```

Simulation Results

- Functional verification shows correct **Sum** and **Carry** for all test vectors.
- Behavioral and Dataflow models give identical outputs.
- Structural model shows **propagation delay** due to carry chain.

8. Synthesis & ASIC Design Metrics (Expected)

Metric	Behavioral	Dataflow	Structural
Area (Gates / LUTs)	Optimized	Optimized	Highest
Power Consumption	Lowest	Medium	Highest
Critical Path Delay	Tool-dependent	Medium	High (Ripple Carry)
Timing Accuracy	Low	Medium	High

Critical Path

Cin → FA0 → FA1 → FA2 → FA3 → Cout

Applications

- ALUs
- Microprocessors
- DSP blocks
- Arithmetic logic circuits in ASICs

Conclusion

A 4-bit parallel adder was successfully designed and verified using Behavioral, Dataflow, and Structural modeling styles. Structural modeling provides accurate power, area, and delay estimation, making it suitable for ASIC workflows, while behavioral modeling is ideal for rapid functional verification.

Experiment No: 3

Title: Design, Synthesis and Simulation of 4-bit Asynchronous Up and Down Counter

Aim: Design, simulate and (where applicable) synthesize a 4-bit asynchronous up and down counter with asynchronous reset. Verify functional behaviour for up and down counting, observe ripple propagation (timing), and discuss synthesis limitations, advantages and disadvantages.

Theory:

Counter types

- **Asynchronous counter:** An asynchronous counter—also called a ripple counter—is a digital counter where only the first flip-flop receives the external clock, and each subsequent flip-flop is triggered by the output of the previous one, creating a cascaded "ripple" effect.
- **Synchronous counter:** All flip-flops receive the same clock; next-state logic drives D/T inputs. Preferred for high-speed/synthesizable designs.

2. Up and Down operation

- **Up counting:** A higher bit toggles when all lower bits are '1' (i.e., a carry). In ripple implementation this is achieved by feeding the previous stage's Q as the clock to the next stage.
- **Down counting:** A higher bit toggles when all lower bits are '0' (i.e., a borrow). In ripple implementation one common technique is to clock the next stage on the *opposite* edge (use previous Q inverted or use negative edge) so toggling occurs when lower bits transition accordingly.

3. Key signals

- clk : primary clock signal (drives LSB).
- rstn : active LOW asynchronous reset (clears counter).

4. Limitations of asynchronous counters

- Not ideal for high frequency due to cumulative propagation delays.

- Use of internal signals as clocks (cascading Q to clock inputs) is not recommended for synthesis on many flows—works for simulation and educational demonstration, but synchronous design is recommended for implementation.

For DOWN counting the higher FFs may be clocked on the *opposite* edge of previous Q (i.e., negedge) so that toggling occurs when lower bits go from 0→1 or 1→0 appropriately.

Advantages:

- Very simple to design.
- Requires fewer gates than synchronous counters.
- Useful for frequency division and low-speed counting.

Disadvantages:

- More prone to decoding errors due to ripple transitions.
- Not suitable for high-precision timing.

Verilog Code

1) Asynchronous 4-bit Up Counter — (Simulation/educational)

```
// async_up.v
// 4-bit asynchronous up counter
// WARNING: uses FF outputs as clocks for higher-stage FFs. Good for simulation/demo only.

module async_counter (
input clk,
input rstn,
output reg [3:0] Count );

always@(negedge clk or negedge rstn) begin
if (!rstn)
Count<= 4'b0000;
else if ( Count==4'd9)
Count<= 4'b0000;
```

```
else
Count<=Count+1;
end
endmodule
```

Testbench

```
module tb_async_counter;
reg Clk, rstn;
wire [3:0] Count;
async_counter uut(.Clk(Clk), .rstn(rstn), .Count(Count));
initial begin
Clk=0;
forever #5 Clk=~Clk;
end
initial begin
rstn=0;
#12 rstn=1;
#200 $finish;
end
endmodule
```

2) Asynchronous 4-bit Down Counter — (Simulation/educational)

```
module async_downc (
input clk,
input rst,
output reg [3:0] Count );
```

```
always@(negedge clk or posedge rst) begin
if (rst)
Count<= 4'd9;
else if ( Count==4'd0)
Count<= 4'd9;
else if (Count==4'd0)
Count<=4'd9;
else
Count<=Count- 4'd1;
end
endmodule
```

Testbench

```
module tb_async_downc;
reg Clk, rst;
wire [3:0] Count;
async_downc uut(Clk,rst,Count);
always #5 Clk=~Clk;
initial begin
Clk=0;
rst=1;
#10 rstn=0;
#100 $finish;
end
endmodule
```

Design Steps

1. **Specification:** Define features — 4 bits, up/down control, asynchronous reset, wrap-around behavior.
2. **Choose design style:** For demonstration create *asynchronous* design; for implementation use *synchronous* design.
3. **Write RTL code:** Implement both versions if you wish to compare.
4. **Create Testbench:** Stimulate clock, reset, enable, and direction; log q.
5. **Functional Simulation (RTL):** Run in ModelSim/Questa/iverilog — verify sequence using waveform viewer or vcd.
6. **Synthesis (synchronous design):** Use Synopsys/Vivado/Genus — examine reports (area, timing) and check for inferred arithmetic operators mapping.
7. **(Optional) Gate-level simulation & STA:** For ASIC flow, run P&R and STA; evaluate maximum clock frequency. Note asynchronous design will show ripple delays but is typically not used for high-frequency operation.
8. **Analysis:** Capture timing of ripple (propagation from LSB to MSB), observe glitches, measure round-trip time and maximum toggling frequency safe for ripple design.
9. **Report:** Include waveform screenshots, tabulated counts, synthesis reports and discussion.

Expected Output

UP sequence (starting from 0000):

0000 -> 0001 -> 0010 -> 0011 -> 0100 -> 0101 -> 0110 -> 0111 -> 1000 -> 1001 -> 1010 -> 1011 -> 1100 -> 1101 -> 1110 -> 1111 -> 0000 -> ...

DOWN sequence (starting from 0000, wrapping):

0000 -> 1111 -> 1110 -> 1101 -> 1100 -> ... -> 0001 -> 0000 -> ...

Conclusion

The **asynchronous counter** is easy to implement and useful for low-speed applications and teaching. However, due to cumulative propagation delays, it is unsuitable for high-speed designs. Using internal signals as clocks makes it unsuitable for many synthesis flows.

Oral / Viva Questions

1. Explain the difference between asynchronous and synchronous counters.
2. How does an asynchronous up counter work? Describe the toggle sequence.
3. Why is ripple propagation delay a problem? How does it affect maximum clock frequency?
4. Why is using flip-flop outputs as clocks not recommended for synthesis?

Experiment No: 4

Title: Design, Synthesis and Simulation of 4-bit Synchronous Up and Down Counter

Aim: Design, simulate and (where applicable) synthesize a 4-bit synchronous up and down counter with synchronous reset. Verify functional behaviour for up and down counting, observe ripple propagation (timing), and discuss synthesis limitations, advantages and disadvantages.

Theory:

Counter types

- **Asynchronous counter:** An asynchronous counter—also called a ripple counter—is a digital counter where only the first flip-flop receives the external clock, and each subsequent flip-flop is triggered by the output of the previous one, creating a cascaded "ripple" effect.
- **Synchronous counter:** All flip-flops receive the same clock. All flip-flops toggle in parallel eliminating ripple delays. Preferred for high-speed/synthesizable designs.

2. Up and Down operation

- **Up counting:** A higher bit toggles when all lower bits are '1' (i.e., a carry). In ripple implementation this is achieved by feeding the previous stage's Q as the clock to the next stage.
- **Down counting:** A higher bit toggles when all lower bits are '0' (i.e., a borrow). In ripple implementation one common technique is to clock the next stage on the *opposite* edge (use previous Q inverted or use negative edge) so toggling occurs when lower bits transition accordingly.

3. Key signals

- clk : primary clock signal (drives LSB).
- rst : synchronous reset (clears counter).

4. Limitations of synchronous counters

- These counters require more complex circuitry.

- Synchronous counters can consume more power due to additional logic gates required to manage simultaneous triggering of all flip-flops.

Advantages:

- High speed.
- Low propagation delay.
- High reliability.

Disadvantages:

- Requires more complex combinational logic to control toggling.
- Slightly higher power consumption.

Verilog Code

1) Synchronous 4-bit Up Counter — (Simulation/educational)

```
// sync_up.v
// 4-bit synchronous up counter
// WARNING: uses FF outputs as clocks for higher-stage FFs. Good for simulation/demo only.

module sync_up ( clk, rst,Count);
input clk;
input rst;
output reg [3:0] Count ;

always@(posedge clk)
 begin
if (rst)
Count= 0;
else Count= Count+1'b1;

end
endmodule
```

Testbench

```
module tb_sync_up;
reg clk, rst;
wire [3:0] Count;
sync_up DUT (clk,rst,Count);
always #5 Clk=~Clk;
initial
begin
clk=0;
rst=0;
#23 rst=1;
#23 rst=0;
end
initial begin
rstn=0;
#200 $finish;
end
endmodule
```

2) Synchronous 4-bit Down Counter — (Simulation/educational)

```
module sync_down ( clk, rst,Count);
input clk;
input rst;
output reg [3:0] Count ;
```

```
always@(posedge clk)
 begin
if (rst)
Count<= 4'b1111;
else Count<= Count-1'b1;
end
endmodule
```

Testbench

```
module tb_sync_down;
reg clk, rst;
wire [3:0] Count;
sync_down DUT (clk, rst, Count);
always #5 Clk=~Clk;
initial
begin
clk=0;
rst=1;
#10 rst=0;
end
initial begin
#200 $finish;
end
endmodule
```

Design Steps

Subject: ASIC Design Lab

1. **Specification:** Define features — 4 bits, up/down control, asynchronous reset, wrap-around behavior.
2. **Choose design style:** For demonstration create *asynchronous* design; for implementation use *synchronous* design.
3. **Write RTL code:** Implement both versions if you wish to compare.
4. **Create Testbench:** Stimulate clock, reset, enable, and direction; log q.
5. **Functional Simulation (RTL):** Run in ModelSim/Questa/iverilog — verify sequence using waveform viewer or vcd.
6. **Synthesis (synchronous design):** Use Synopsys/Vivado/Genus — examine reports (area, timing) and check for inferred arithmetic operators mapping.
7. **(Optional) Gate-level simulation & STA:** For ASIC flow, run P&R and STA; evaluate maximum clock frequency. Note asynchronous design will show ripple delays but is typically not used for high-frequency operation.
8. **Analysis:** Capture timing of ripple (propagation from LSB to MSB), observe glitches, measure round-trip time and maximum toggling frequency safe for ripple design.
9. **Report:** Include waveform screenshots, tabulated counts, synthesis reports and discussion.

Expected Output

UP sequence (starting from 0000):

0000 -> 0001 -> 0010 -> 0011 -> 0100 -> 0101 -> 0110 -> 0111 -> 1000 -> 1001 -> 1010 -> 1011 -> 1100 -> 1101 -> 1110 -> 1111 -> 0000 -> ...

DOWN sequence (starting from 0000, wrapping):

0000 -> 1111 -> 1110 -> 1101 -> 1100 -> ... -> 0001 -> 0000 -> ...

Conclusion

The **synchronous counter** is easy to implement and useful for fast operation due to no propagation delay. The synchronous counters have stable performance. They have precise timing all flip-flops switch together.

Oral / Viva Questions

1. Explain the difference between asynchronous and synchronous counters.
2. How does a synchronous up counter work?
3. What are the advantages and diadvantages of synchronous counter?
4. Why is using flip-flop outputs as clocks not recommended for synthesis?

Experiment No. 5

Title: Design, Synthesis and Simulation of RAM/ROM using Verilog/VHDL (Behavioral Modeling)

Aim: To design, simulate and synthesize RAM and ROM memory modules using Verilog/VHDL behavioral modeling and analyze hardware parameters such as LUT utilization, power, critical timing and delay estimation.

Objectives:

- To understand the concept of memory design in HDL.
- To implement RAM and ROM using behavioral modeling.
- To perform RTL simulation and synthesis using FPGA design tools.
- To analyze resource utilization (LUTs), power consumption and timing delays.

Theory

Memory plays a crucial role in computing devices, ensuring smooth data processing and storage. Among the various types of memory, Static Random-Access Memory (SRAM) and Dynamic Random-Access Memory (DRAM) are two fundamental categories. While both serve as volatile memory in computing systems, they have significant differences in terms of design, performance, and application. In this blog, we will delve into the core distinctions between SRAM and DRAM.

RAM (Random Access Memory)

- RAM allows read and write operations.
- Data can be accessed randomly using address lines.
- Commonly used for temporary data storage.

Static Random-Access Memory (SRAM) is a type of volatile memory that stores data using flip-flop circuits. Unlike DRAM, which requires periodic refreshing, SRAM maintains stored data as long as power is supplied.

- Uses flip-flop circuits to store data.
- Does not require refreshing to retain data.
- Offers faster access time and lower latency.
- Consumes more power compared to DRAM.

- More expensive due to complex architecture.
- Typically used in cache memory (L1, L2, L3), registers, and high-performance applications.

Dynamic Random-Access Memory (DRAM) stores data using capacitors and transistors. Unlike SRAM, the charge in capacitors leaks over time, requiring constant refreshing to maintain data.

- Stores data using capacitors and transistors.
- Requires regular refreshing to retain data.
- Slower than SRAM but still fast enough for main memory applications.
- Consumes less power compared to SRAM.
- More cost-effective and has higher storage density.
- Commonly used in main memory (RAM) of computers, laptops, and other devices.

Both SRAM and DRAM play essential roles in modern computing. SRAM is preferred for high-speed operations, while DRAM is widely used for main memory due to its cost-effectiveness and higher density. Understanding their differences helps in selecting the right type of memory for different applications, ensuring optimal system performance.

ROM (Read-Only Memory)

It is a non-volatile memory was used to operate the system. As its name refers to read-only memory, we can only read the stored programs and data.

- Information stored in ROM is permanent.
- Information and programs are stored on ROM in binary format (0s and 1s).
- It is used in the start-up process of the computer.

Behavioral Modeling

Behavioral modeling describes how the system works without focusing on gate-level implementation. It uses high-level constructs like always blocks, processes and case statements.

Advantages:

- Easy to write
- Faster simulation
- Good for large system modeling

Tools Required

- Xilinx Vivado / Xilinx ISE / ModelSim / Quartus
- Verilog / VHDL HDL
- FPGA development board (optional)

Verilog Code

A) RAM Design (8-bit Data, 64B Memory)

```
module single_port_ram(
input clk, we,
input [5:0] addr,                 // this is address;
input [7:0] data,                 // this is input data;
output [7:0] q                    // q is output data; 8 bit data;
);
reg [7:0] ram [63:0];
reg[5:0] addr_reg;
always @(posedge clk)
begin
if(we)
ram[addr] <= data;
else
addr_reg<= addr;
end
assign q= ram[addr_reg];
endmodule
```

B) ROM Design (Behavioral)

```
module rom (
input [3:0] addr,
output reg [7:0] data
```

```
);
always @(*)
begin
case(addr)
4'b0000: data = 8'h11;
4'b0001: data = 8'h22;
4'b0010: data = 8'h33;
4'b0011: data = 8'h44;
4'b0100: data = 8'h55;
4'b0101: data = 8'h66;
4'b0110: data = 8'h77;
4'b0111: data = 8'h88;
default: data = 8'h00;
endcase
end
endmodule
```

6. Testbench for RAM

```
module single_port_ram_tb( );
reg clk;
reg we;
reg [5:0] addr;
reg [7:0] data;
wire [7:0] q;
single_port_ram SUT(
.clk(clk),
.we(we),                 // 'we' is write & read control. High is write, low is read;
.addr(addr),             //  this is address;
.q(q),                   //   q is output data; 8 bit data;
```

```
.data(data)
);
initial begin
clk=1'b1;
forever #50 clk=~clk;
end
// we will write 3 data into the address 0,1,2, then read it out;
initial begin
data=8'h01;
addr=5'd0;
we=1'b1;
#100;
data=8'h02;
addr=5'd01;
#100;
data=8'h03;
addr=5'd02;
#100;
// now begin to read it;
addr= 5'd0;                 // address 0;
we=1'b0;                    // read address 0, it should be data 01;
#100;
addr= 5'd1;                 // read address 1, it should be data 02;
#100;
addr= 5'd2;                 // read address 2, it should be data 03;
#100;
end
endmodule
```

Simulation Procedure

Open Xilinx Vivado / ModelSim.

Create a new project.

Add the Verilog source file (RAM/ROM).

Add the testbench file.

Run Behavioral Simulation.

Observe RTL waveform.

Perform Synthesis.

Generate Design Summary.

Note LUT utilization, power report and timing analysis.

Expected Outputs

- RTL schematic of RAM/ROM
- Simulation waveform
- Memory read/write operation
- Design utilization report

Result:

RAM and ROM modules were successfully designed, simulated and synthesized using Verilog behavioral modeling, and hardware parameters such as LUT utilization, timing delay and power consumption were analyzed.

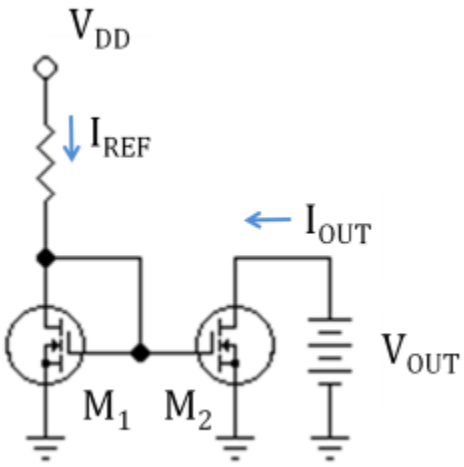
Experiment No: 4

Title: To design, simulate MOS based current mirror circuit and implement voltage-controlled oscillator.

Aim: Design and simulate MOS based current mirror circuit in LTspice . Verify the input and output currents on changing input and output resistance. Design voltage-controlled oscillator in LTspice and observe the output voltages.

Theory:

1. **MOS based current mirror circuit :**

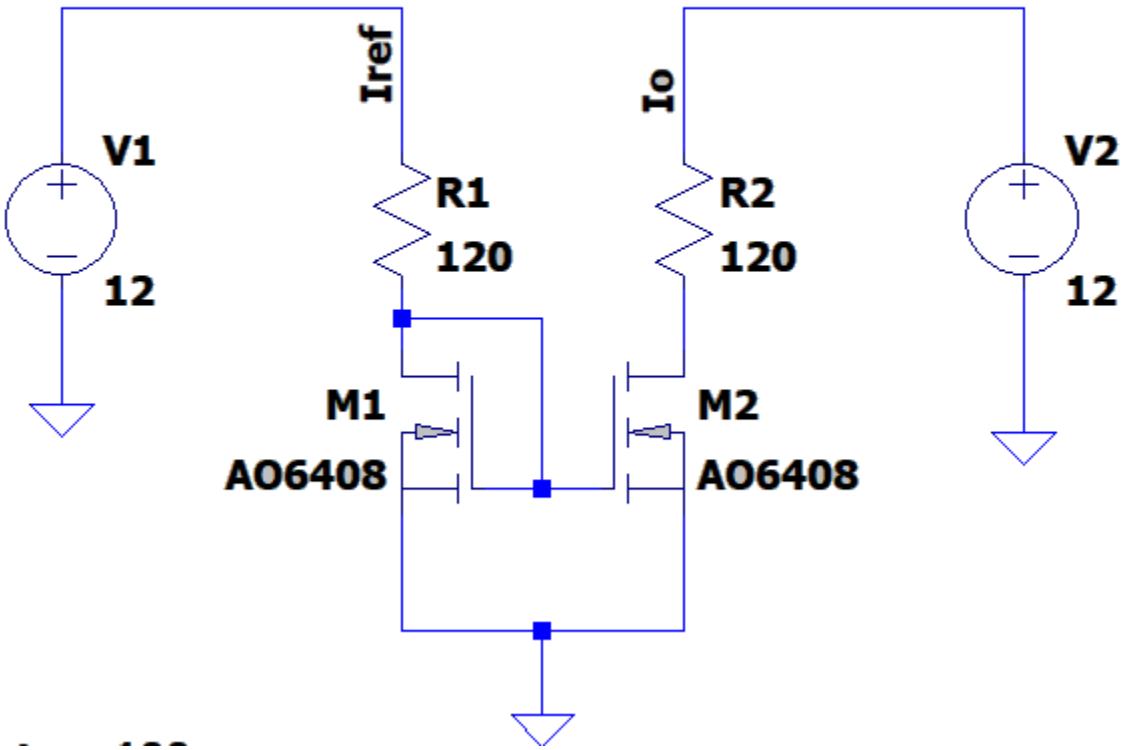


The basic current mirror can be implemented using MOSFET transistors, as shown in Figure. Transistor M_1 is operating in the saturation or active mode, and so is M_2. Here, the output current I_{OUT} is directly related to I_{REF}.

The drain current of a MOSFET I_D is a function of both the gate-source voltage and the drain-to-gate voltage of the MOSFET given by $I_D = f(V_{GS}, V_{DG})$, a relationship derived from the functionality of the MOSFET device. In the case of transistor M_1 of the mirror, $I_D = I_{REF}$. Reference current I_{REF} is a known current, and can be provided by a resistor as shown, or by a "threshold-referenced" or "self-biased" current source to ensure that it is constant, independent of voltage supply variations.

Using V_{DG} = 0 for transistor M_1, the drain current in M_1 is $I_D = f(V_{GS}, V_{DG}=0)$, so we find: $f(V_{GS}, 0) = I_{REF}$, implicitly determining the value of V_{GS}. Thus I_{REF} sets the value of V_{GS}. The circuit in the diagram forces the same V_{GS} to apply to transistor M_2. If M_2 is also biased with zero V_{DG} and provided transistors M_1 and M_2 have good matching of their properties, such as channel length, width, threshold voltage, etc., the relationship $I_{OUT} = f(V_{GS}, V_{DG} = 0)$ applies, thus setting $I_{OUT} = I_{REF}$; that is, the output current is the same as the reference current when V_{DG} = 0 for the output transistor, and both transistors are matched.

Schematic:



2. Voltage-controlled Oscillator:

A **voltage-controlled oscillator** (**VCO**) is an electronic oscillator whose oscillation frequency is controlled by a voltage input. The applied input voltage determines the instantaneous oscillation frequency. Consequently, a VCO can be used for frequency modulation (FM) or phase modulation (PM) by applying a modulating signal to the control input. A VCO is also an integral part of a phase-locked loop. VCOs are used in synthesizers to generate a waveform whose pitch can be adjusted by a voltage determined by a musical keyboard or other input.

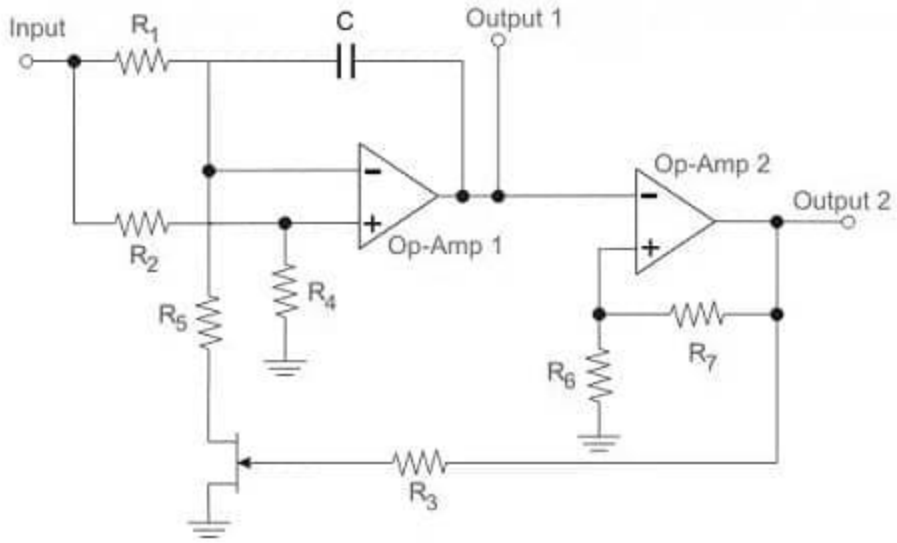
The output waveform of this VCO will be square wave. As we know the output frequency is related to the control voltage. In this circuit the first Op-amp will function as an integrator. The voltage divider arrangement is implemented here. Because of this, the half of the control voltage that is given as input is given to the positive terminal of the Op-amp 1. The same level of voltage is maintained at the negative terminal. This is to sustain the voltage drop across the resistor, R1 as half of the control voltage.

When the MOSFET is in on condition, the current flowing from the R1 resistor passes through the MOSFET. The R2 have half the resistance, same voltage drop and twice the current as that of R1. So, the extra current charges the connected capacitor. The Op-amp 1 should provide a gradually increasing output voltage to supply this current.

When the MOSFET is in off condition, the current flowing from the R1resistor passes through the capacitor, get discharged. The output voltage obtained from the Op-amp 1 at this time will be falling. As a result, a triangular waveform is generated as the output of Op-amp 1.

The Op-amp 2 will operate as Schmitt trigger. The input to this Op-amp is triangular wave which is the output of the Op-amp 1. If the input voltage is higher than the threshold level, the output from the Op-amp 2 will be VCC. If the input voltage is less than the threshold level,

the output from the Op-amp 2 will be zero. Therefore, the output of the Op-amp 2 will be square wave.

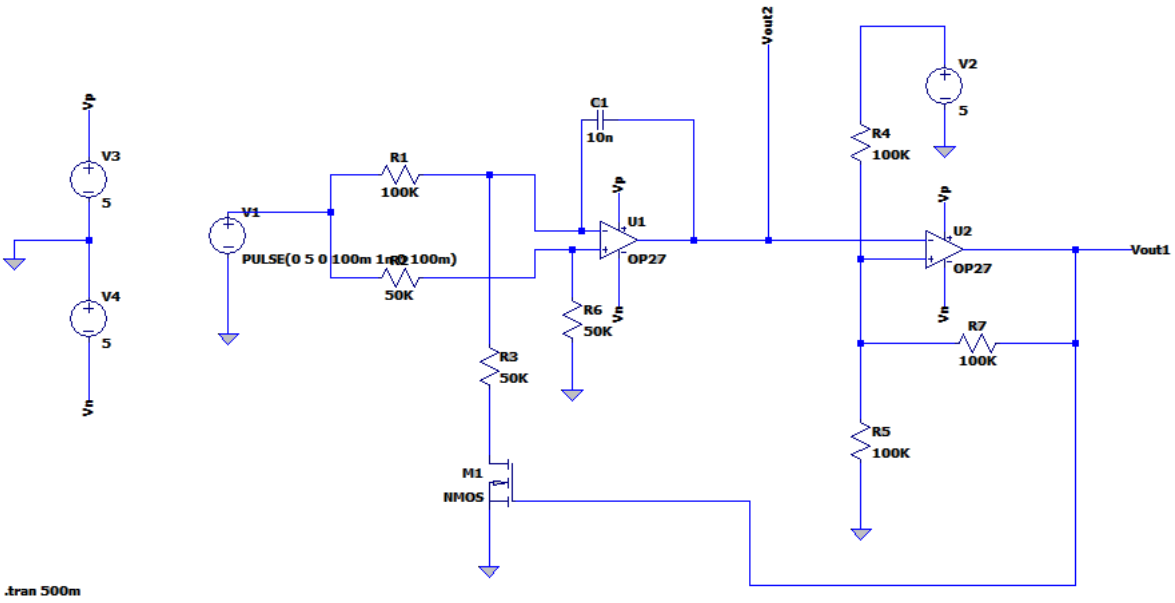


Applications of Voltage Controlled Oscillator

- Function generator
- Phase Locked Loop
- Tone generator
- Frequency-shift keying
- Frequency modulation

Subject: ASIC Design Lab

Schematic:



Procedure:

1. Open a new schematic and place the required components (resistor, capacitor, nmos, op-amp etc.) to set up the circuit.
2. Connect the components.
3. Add voltage source and ground wherever necessary.
4. Set the parameters. Assign required values of capacitance, resistance, voltage source.
5. Run the simulation and observe the input and output.
6. After running the simulation, check the output to ensure the circuit is working appropriately.

Result:

Conclusion:

Oral / Viva Questions

1. When does the current mirror circuit fails ?
2. Explain the working of VCO.
3. What are the function of op-amps used in VCO?
4. Where can VCO be used?